

GitAgile: Technical Specification & Implementation Plan (V2)

Author: Emmanuel Dadem

Version: 2.0.0

Target Tooling: Claude Code & Multi-Agent AI Orchestration

1. Executive Summary & Core Mission

GitAgile is an open-source, local-first, git-native project management tool designed to serve as a complete replacement for licensed enterprise management systems like Jira or Trello. By aligning software planning states directly with codebase development states, GitAgile uses a git-tracked directory (`.agile/`) populated with structured Markdown files as the Single Source of Truth (SSOT).

This architecture eliminates standard developer friction. Developers can manage their backlogs offline, track ticket evolution through standard Git commit logs, and resolve scheduling conflicts natively via branch merges. Furthermore, keeping project requirements inside plain-text repository folders allows software agents—such as Claude Code, GitHub Copilot, and custom codegen pipelines—to analyze features directly and write code without external REST APIs. To guarantee PO (Product Owner) and QA (Quality Assurance) usability, the system includes a zero-config local web browser Kanban board that operates completely on top of local files.

2. Hybrid Architecture & Sync Engine

While Markdown files are perfect for versioning and LLM processing, reading hundreds of flat text files sequentially causes severe performance bottlenecks for visualization. To maintain high-speed UI responsiveness and complex relational linking (e.g., mapping sub-projects, rolling up metrics to Sprints), GitAgile utilizes a hybrid storage model.

- **Git-Tracked Markdown (SSOT):** Structured `.md` files located in subfolders (`.agile/tickets/`, `.agile/epics/`, and `.agile/sprints/`) are tracked in Git.
- **Ephemeral SQLite Cache (Query Index):** A local database located at `.agile/.cache.db`. It is added to the project's `.gitignore` and is never committed. The cache database functions purely as a read index for the CLI, TUI, and local web viewer.

2.1 Bidirectional Sync Engine Mechanics

The sync engine runs in the background during any tool interaction (TUI loading, CLI query, or Web Socket event) and performs the following routines:

1. **Disk-to-Cache Sync (Read-Through):** The engine tracks file modification times (mtime) or calculates quick file hashes inside .agile/. If a Markdown file is newer than the last recorded DB update:
 - Parse the file's YAML frontmatter and markdown body content.
 - Execute a fast SQL INSERT OR REPLACE statement in SQLite.
2. **Cache-to-Disk Sync (Write-Through):** When a state mutation happens (e.g., dragging a ticket card in the Web board or editing status in the TUI):
 - Directly update the local SQLite table row instantly to preserve interface fluidity.
 - In the background, serialize the data model back into the actual Markdown file on disk, retaining existing user formatting, descriptions, and comments.

3. Storage Format & Schema Specifications

3.1 Ticket Markdown Structure

Ticket files must follow a strict, standardized frontmatter structure. Below is an example of an epic-linked story located at .agile/tickets/042-add-email-verification.md:

```
---
id: "042"
title: "Add email verification to signup flow"
type: "task"
status: "IN_PROGRESS"
priority: "HIGH"
assignee: "Emmanuel Dadem"
epic: "user-auth"
sprint: "3"
points: 5
sub_project: "frontend-app"
created_at: "2026-07-15T10:00:00Z"
updated_at: "2026-07-15T15:30:00Z"
---
```

Goal

We need to prevent spam accounts by verifying user emails before they can access their dashboard.

Checklist

- [x] Design verification email template
- [] Implement backend token generator (expires in 24h)
- [] Create UI block state for unverified accounts

Acceptance Criteria

- ****Given**** a user registers with a valid email
- ****When**** they submit the form
- ****Then**** they should receive a link with a secure token
- ****And**** they cannot log in until they click the link

3.2 Relational SQLite Schema

The ephemeral cache SQLite database handles rapid relational evaluations with clean, simple foreign keys:

```
CREATE TABLE epics (  
  id TEXT PRIMARY KEY,  
  title TEXT NOT NULL,  
  status TEXT NOT NULL,  
  description TEXT  
);
```

```
CREATE TABLE sprints (  
  id TEXT PRIMARY KEY,  
  name TEXT NOT NULL,  
  status TEXT NOT NULL, -- FUTURE, ACTIVE, COMPLETED  
  start_date TEXT,  
  end_date TEXT  
);
```

```
CREATE TABLE tickets (  
  id TEXT PRIMARY KEY,  
  title TEXT NOT NULL,  
  type TEXT NOT NULL,  
  status TEXT NOT NULL,  
  priority TEXT NOT NULL,  
  assignee TEXT,
```

```

    epic_id TEXT,
    sprint_id TEXT,
    points INTEGER,
    sub_project TEXT,
    content TEXT, -- Store the raw Markdown body below the frontmatter
    created_at TEXT,
    updated_at TEXT,
    FOREIGN KEY (epic_id) REFERENCES epics(id),
    FOREIGN KEY (sprint_id) REFERENCES sprints(id)
);

```

4. Unified Technology Stack

To distribute GitAgile as a single portable, static binary, the chosen technologies prioritize zero-runtime dependencies on the user's local machine.

Component	Technology Stack	Architectural Rationale
Core CLI Binary	Go (Golang)	Compiles to a single static binary with absolute zero runtime dependencies. Provides top-tier file system operations, speed, and concurrency handling.
Interactive TUI	Go Bubbletea (Charm CLI)	Enables highly-responsive terminal interfaces following an Elm-architecture pattern. Ideal for mouse-free keyboard warriors.
Index & Cache	SQLite (ephemeral, local)	Eliminates database server overhead. Solves complex relational queries instantly without degrading the simple Markdown-based single

Component	Technology Stack	Architectural Rationale
		source of truth.
Web UI Core	React, TypeScript, Vite	Provides a lightweight, performant SPA codebase. Compiled static assets embed seamlessly into the Go binary via go:embed.
Local Server	Go Net/HTTP + WebSockets	Establishes real-time, bi-directional sync from terminal edits straight to browser canvases via standard sockets.

5. Functional Requirements & Integration Workflows

5.1 Command Line Interface (CLI) Engine

- `git-agile init`: Instantiates the local `.agile/` subfolders, establishes templates, and appends the `.cache.db` to `.gitignore`.
- `git-agile board`: Opens an interactive Bubbletea terminal dashboard allowing developers to view active sprints, filters, and drag cards via terminal hotkeys.
- `git-agile ui`: Launches a lightweight web server on `localhost:1337`, mounting the embedded React/Tailwind visual Kanban and opening the system browser automatically.
- `git-agile task create "Title" [options]`: Fast terminal input generation supporting arguments like `--assignee`, `--points`, and `--epic`.

5.2 Automated Git Integrations

GitAgile bridges code changes with planning states via standard Git Hooks (installed during `git-agile init`):

- **Branch Hook (post-checkout)**: If a branch matches `feature/042-add-verification`, the hook triggers a quick evaluation to move Ticket 042's status YAML field to `IN_PROGRESS`.
- **Commit Hook (commit-msg)**: Evaluates if a commit message references active ticket patterns. If verified, updates the ticket markdown's status log and registers the commit

SHA dynamically inside the ticket's history segment.

6. Native AI Orchestration Blueprint

GitAgile is built to exploit LLM capabilities natively. The CLI can directly assemble rich context packets for AI execution.

6.1 Interfacing with Claude Code

When developers use Claude Code to implement features, they need to supply the exact requirements. By running `git-agile ai prompt [id]`, the tool automatically reads the target ticket markdown, matches it against file paths declared in `sub_project`, and prints a specialized prompt ready to copy-paste or pipe into Claude:

Analyze and resolve Ticket #042 in our repository.

Context details:

Title: Add email verification to signup flow

Objective: Ensure account safety by verifying emails via token links.

Acceptance Criteria:

- Verify token expires in 24h
- Block login state for unverified accounts

Associated sub-project files:

1. `frontend-app/src/auth/signup.js`
2. `frontend-app/src/db/schema.js`

Perform modifications to satisfy the criteria while maintaining code health.

6.2 Embedded LLM Features

If provided with a local Ollama host or an API key (Anthropic/OpenAI), the GitAgile engine exposes:

- `git-agile draft "Brief concept summary"`: The LLM analyzes the concept, creates an Epic markdown file, structures nested sub-tasks, and generates realistic Gherkin acceptance tests.
- `git-agile review`: Scans all Git commits merged during the active Sprint, correlates them back to completed Markdown tickets, and synthesizes a high-level, human-readable

Sprint Review summary detailing delivered milestones and roll-over estimations.

7. Security, Conflict Management, & Scalability

Collaborative git repositories inevitably encounter merge conflicts. GitAgile's file strategy proactively limits these collisions:

- **Single-File Isolation:** Every ticket has an independent markdown file. Two developers will only trigger conflicts if they are modifying the exact same ticket concurrently.
- **Conflict Solver:** For conflicts in multi-user planning configurations (e.g., epic planning changes), the TUI exposes a git-agile resolve [id] tool that visualizes conflicts visually and helps write cleanly integrated YAML back to disk.

8. Collaboration, Synchronization & Multi-Machine Hosting Architecture

To bridge the gap between local developers and non-technical stakeholders (Product Owners, Agile Coaches, QA Managers) operating on separate machines, GitAgile supports dual collaboration topologies. This ensures real-time visualization and management of the sprint board regardless of the team's operational environment.

8.1 Mode A: Serverless Git-Native Sync (Local-First)

Designed for autonomous teams that prefer complete data privacy, serverless operation, and offline support:

- **Asynchronous Commits:** Developers mutate files on their machines via CLI/TUI or local Web UI. Code branches and corresponding modified .md files are pushed to the remote repository (GitHub/GitLab) as a standard part of their pull request loop.
- **Background Sync Daemon:** A lightweight system service running git-agile daemon periodically polls the remote via git fetch. When newer ticket markdown files are pulled down to a PO's or colleague's system, the engine triggers a Disk-to-Cache index refresh, and active browser cards update instantly.

8.2 Mode B: Central Shared Server Mode (Live Web Portal)

Designed for organizations where Product Owners, Business Analysts, and Stakeholders require a single centralized "real-time" dashboard without installing local command-line binaries:

- **Centralized Deployment:** The Go CLI binary runs as a centralized web service on a company VPS or cloud instance (e.g., AWS, GCP, or a private server) using the command `git-agile serve --port 80`.

- **Webhook Pull Actions:** The Git remote host (e.g., GitHub, GitLab) is configured with a webhook pointing to the central server. The moment a developer pushes branch changes, the webhook triggers the central server to pull changes instantly.
- **Real-Time Broadcasts:** Upon pulling changes, the central server updates its shared SQLite engine and instantly broadcasts UI state updates to connected PO browser clients over persistent WebSockets connections.

9. Incremental Implementation Plan for Claude Code

When initiating the codebase construction with Claude Code, execute the stages sequentially:

1. **Phase 1 (Disk Operations):** Build a robust parser in Go that can cleanly serialize and deserialize YAML frontmatter and standard Markdown body segments. Establish the directory layouts inside `.agile/`.
2. **Phase 2 (SQLite Database Sync):** Set up the database adapters and sync loop. Build the differential filesystem scanner to monitor mtime and mirror folder changes directly to SQLite.
3. **Phase 3 (CLI Structure):** Structure the Cobra CLI framework commands (`init`, `task`, `sync`) and handle local path mapping.
4. **Phase 4 (Local Web UI Server):** Develop the Vite/TypeScript React SPA (using standard Kanban cards and board columns). Bundle the static files into the compiled Go binary using `go:embed` and wire up the WebSockets server.
5. **Phase 5 (Collaboration Mode):** Implement the `git-agile serve` command structures. Set up webhook ingestion to pull files asynchronously and trigger state broadcasts down to WebSockets browser clients.
6. **Phase 6 (Interactive TUI):** Create the Bubbletea dashboard for quick developer command states.
7. **Phase 7 (AI Agents):** Implement prompt generation layouts and configure adapters to call local or cloud models for drafting sprints and generating reviews.